



CORE CS (THEORY)

By Abhishek Rathor

Instagram: [code.abhii07](#) (SYNTAX ERROR)

❖ Top 50 Tricky Core CS Questions

1. Why is Binary Search faster than Linear Search?

Binary search reduces the search space by half each step. It works only on sorted data. This reduces time complexity to $O(\log n)$. Linear search checks elements one by one, making it slower.

2. Why is Stack faster than Array?

Stack operations happen at one end only. There is no shifting of elements. Access is restricted, which reduces overhead. This makes stack operations faster.

3. Why is Hashing faster than Searching?

Hashing directly maps keys to indexes. It avoids scanning elements. Average time complexity is $O(1)$. Performance depends on hash function.

4. Why is Linked List preferred over Array sometimes?

Linked list has dynamic size. Insertion and deletion are easier. No shifting required. Arrays waste memory when resized.

5. Why is Recursion memory expensive?

Each recursive call uses stack memory. Function calls remain until base case. This increases memory usage. May cause stack overflow.

6. Why is Binary Search Tree better than Array?

BST allows faster insertion and deletion. Searching is $O(\log n)$ on average. Arrays need shifting. BST maintains sorted structure.

7. Why is Virtual Memory needed?

Physical memory is limited. Virtual memory uses disk as extension. Allows large programs to run. Improves memory utilization.

8. Why is Deadlock difficult to detect?

Deadlock depends on resource state. System must check circular wait. Requires overhead. Detection is complex in large systems.

9. Why does Context Switching reduce performance?

Context switching saves and restores state. CPU does not perform useful work. Too many switches waste time. Increases overhead.

10. Why is Multithreading faster than Multiprocessing?

Threads share memory. Communication is faster. Process switching is heavy. Threads reduce overhead.

11. Why is TCP slower than UDP?

TCP ensures reliability. Uses acknowledgements and error control. UDP skips checks. Hence UDP is faster.

12. Why is Normalization not always preferred?

Normalization increases joins. Joins reduce performance. Denormalization improves speed. Used in large databases.

13. Why does Indexing improve performance?

Index reduces data scanning. It provides quick access path. Similar to book index. Improves query speed.

14. Why is NULL not equal to zero?

NULL represents unknown value. Zero is a valid value. NULL comparisons need special handling. They are fundamentally different.

15. Why does SQL WHERE not allow aggregate functions?

WHERE filters rows before grouping. Aggregate functions work after grouping. HAVING is used instead. Order of execution matters.

16. Why is Primary Key important?

It ensures uniqueness. Prevents duplicate records. Improves indexing. Maintains data integrity.

17. Why is Foreign Key required?

It maintains relationship between tables. Prevents invalid references. Ensures consistency. Enforces constraints.

18. Why is Cache Memory faster than RAM?

Cache is closer to CPU. Smaller size but faster access. Uses high-speed memory. Reduces access time.

19. Why is Compiler faster than Interpreter?

Compiler translates entire code once. No repeated translation. Execution is faster. Interpreter translates line by line.

20. Why does Binary Search require sorted array?

Binary search divides array logically. Comparison depends on order. Unsorted data breaks logic. Sorting is mandatory.

21. Why is $O(\log n)$ better than $O(n)$?

Logarithmic growth is slower. Handles large input efficiently. Linear grows faster. Better scalability.

22. Why does Stack Overflow occur?

Excessive recursive calls. No proper base case. Stack memory fills up. Program crashes.

23. Why is Paging better than Segmentation?

Paging avoids external fragmentation. Fixed-size blocks. Easier memory management. Segmentation causes fragmentation.

24. Why is OS needed?

Manages hardware resources. Controls processes and memory. Provides user interface. Essential for system operation.

25. Why is Kernel Mode risky?

Kernel mode has full access. Errors affect entire system. Needs careful handling. Used only by OS.

26. Why is Synchronization required?

Multiple processes share data. Without control, data becomes inconsistent. Synchronization ensures correctness. Prevents race conditions.

27. Why does Race Condition occur?

Processes access shared data simultaneously. No proper locking. Execution order varies. Leads to unpredictable results.

28. Why is Semaphore better than Busy Waiting?

Semaphore blocks processes. Busy waiting wastes CPU. Semaphore improves efficiency. Reduces CPU usage.

29. Why is Cloud Computing cost-effective?

No hardware investment. Pay-as-you-go model. Scalable resources. Reduces maintenance cost.

30. Why is Agile preferred over Waterfall?

Agile is flexible. Supports changes easily. Continuous feedback. Faster delivery.

31. Why is Sorting needed before Binary Search?

Binary search depends on order. Sorted data enables division. Unsorted breaks logic. Sorting is prerequisite.

32. Why is Graph traversal complex?

Graphs have cycles. Multiple paths exist. Need visited tracking. More complex than trees.

33. Why is Tree traversal easier than Graph?

Trees have no cycles. Single path between nodes. Simple traversal. Less complexity.

34. Why is Stack used in Function Calls?

Function calls follow LIFO. Last call executes first. Stack maintains order. Simplifies execution.

35. Why is Queue used in Scheduling?

Queue follows FIFO. Fair processing. Easy implementation. Used in CPU scheduling.

36. Why is Heap used in Priority Queue?

Heap maintains priority order. Fast insertion and deletion. Efficient access to highest priority. Used in scheduling.

37. Why is Encryption necessary?

Protects data from attackers. Ensures confidentiality. Prevents unauthorized access. Essential for security.

38. Why is DNS needed?

Humans remember names. Computers use IP addresses. DNS translates names. Simplifies internet use.

39. Why is Load Balancing important?

Distributes traffic evenly. Prevents overload. Improves availability. Enhances performance.

40. Why is Distributed System complex?

Multiple nodes involved. Synchronization needed. Network failures possible. Hard to manage consistency.

41. Why is Testing important?

Detects defects early. Improves quality. Reduces failure risk. Ensures reliability.

42. Why is Version Control important?

Tracks code changes. Enables collaboration. Prevents loss. Maintains history.

43. Why is Data Integrity critical?

Ensures correct data. Prevents corruption. Maintains trust. Essential for systems.

44. Why is Modular Programming better?

Breaks system into modules. Easy maintenance. Reusable code. Improves readability.

45. Why is Abstraction important?

Hides complexity. Improves usability. Focuses on essentials. Reduces errors.

46. Why is Polymorphism powerful?

Same interface, different behavior. Improves flexibility. Supports extensibility. Reduces code duplication.

47. Why is Inheritance risky?

Tight coupling. Changes affect child classes. Increases complexity. Needs careful design.

48. Why is Encapsulation needed?

Protects data. Controls access. Improves security. Maintains integrity.

49. Why is Core CS important for interviews?

Tests fundamentals. Shows problem-solving ability. Required for design thinking. Used in all tech roles.

50. Why do interviewers ask tricky CS questions?

They test conceptual clarity. Check depth of knowledge. Evaluate thinking skills. Distinguish strong candidates.

Connect & Share:

Tag us on your success stories:

- **Instagram:** [@code.abhii07](#)
- **YouTube:** [SYNTAX ERROR](#)